

Zybo Z7 DMA Audio Processing Project

Project Overview

This project uses the Zybo Z7 DMA Audio Codec for a hardware/software co-design project for audio Processing. The hardware allows recording of the audio through the Zybo Z7 audio codec, stores the captured audio in DDR memory, and plays it back through the audio output. The design allows audio to be recorded from either the line in or the mic in jack located on the Zybo, then moves the data into DDR using AXI DMA, converts the raw DMA samples into signed 16-bit PCM audio, downsamples the audio to 16 kHz, and passes a one-second audio window into software module. At the present stage, the module includes a working voice activity placeholder that correctly distinguishes quiet recordings from speech recordings. The project structure is prepared for future TensorFlow Lite Micro or custom embedded ML model integration.

This repository is intended to satisfy an ECE 520/L final project requirement by including both hardware and software components and demonstrating a hardware/software co-design process on a Xilinx/AMD SoC development board.

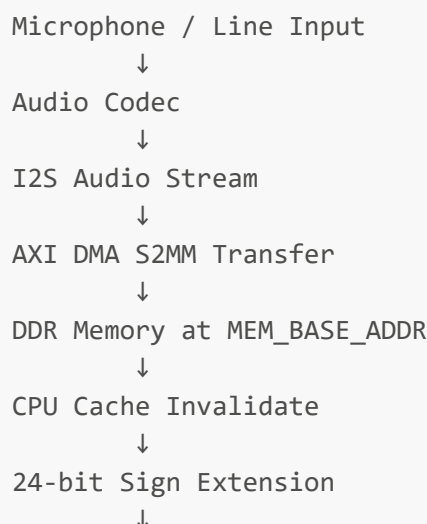
Course Submission Context

This project was developed for California State University, Northridge ECE 520/L: System-on-Chip Design with Lab. The final project guidelines require the project to use an FPGA or SoC topic, include both hardware and software components, run on a Xilinx/AMD SoC development board such as a Zybo or Zedboard, and include a GitHub repository with a README file and project files. The guidelines also require an instruction document, final report, video demonstration, and citations for references used.

System Architecture

The system is built around the Zybo Z7 board, which contains a Xilinx Zynq-7000 SoC. The project uses both the Processing System (PS) and Programmable Logic (PL). The PL-side audio path and DMA logic are used to move audio data efficiently, while the PS-side software controls recording, configures peripherals, reads the DMA buffer, preprocesses audio, and runs the keyword-spotting logic.

High-level signal flow:



```

16-bit PCM Conversion
    ↓
96 kHz to 16 kHz Downsampling
    ↓
1-second kws_input[16000] Buffer
    ↓
KWS_Run() Keyword Spotting Module
    ↓
UART Output: silence / speech detected / future keyword result

```

Hardware Design and Processing Logic

The hardware design is based on the Zybo Z7 DMA audio system. The important hardware and processing components are:

- **Zynq Processing System (PS):** Runs the bare-metal Vitis software application.
- **Audio Codec:** Captures microphone or line-in audio and produces digital audio samples.
- **I2C Interface:** Configures the audio codec registers before recording and playback.
- **I2S / Audio Stream Interface:** Carries the actual digital audio samples between the codec/audio logic and the DMA stream path.
- **AXI DMA:** Transfers audio samples between the streaming audio hardware and DDR memory.
- **DDR Memory:** Stores the recorded audio buffer at `MEM_BASE_ADDR`.
- **Interrupt Controller:** Receives DMA completion interrupts so the software knows when recording or playback has finished.
- **UART:** Displays debug messages, audio statistics, and keyword-spotting results.

The recording path uses the DMA S2MM direction, meaning stream-to-memory-mapped. The audio stream is written into DDR memory beginning at `MEM_BASE_ADDR`. Once the DMA transfer finishes, an interrupt sets the software completion flag. The software then stops the stream, invalidates the cache for the DMA-written memory range, and reads the audio data for preprocessing.

I2C and interrupts are both part of the processing logic, but they serve different purposes. I2C is used for codec configuration and control. Interrupts are used for DMA completion notification and software control flow. The audio samples themselves are moved through the I2S/audio stream and AXI DMA path, not through I2C.

Software Design and Processing System

The software is a bare-metal Vitis application. The major software modules are:

<code>demo.c</code>	Main application flow, button handling, DMA event handling, audio preprocessing call sequence
<code>audio.c</code>	Audio codec and audio stream control functions
<code>dma.c</code>	AXI DMA initialization and transfer support
<code>iic.c</code>	I2C configuration support for the audio codec
<code>intc.c</code>	Interrupt controller setup and interrupt handlers
<code>userio.c</code>	User input/output support such as buttons, switches, LEDs, or UART prompts

kws.cc	Keyword-spotting module written in C++ for future ML integration
kws.h	C-compatible interface for KWS_Init() and KWS_Run()

The software flow after a recording completes is:

1. Detect DMA S2MM completion through the interrupt/event flag.
2. Stop the I2S stream and transfer control logic.
3. Invalidate the CPU data cache for the DMA-written DDR range.
4. Read the raw DMA words from `MEM_BASE_ADDR`.
5. Convert raw 32-bit DMA words into signed 16-bit PCM samples.
6. Extract a one-second 16 kHz audio window into `kws_input`.
7. Call `KWS_Run(kws_input, kws_count)`.
8. Print the result over UART.

Audio Data Conversion

The raw DMA buffer contains 32-bit words, but the actual audio sample is stored in the lower 24 bits. The upper 8 bits are not part of the audio sample. Therefore, each raw sample must be converted before it can be used for keyword spotting.

Example raw word:

```
0x00FDDD13
```

The lower 24 bits are:

```
0xFDDD13
```

Because the audio is signed 24-bit data, negative values must be sign-extended into a 32-bit signed integer. The project uses logic equivalent to:

```
static int32_t SignExtend24(u32 raw)
{
    int32_t sample = raw & 0x00FFFFFF;

    if (sample & 0x00800000)
    {
        sample |= 0xFF000000;
    }

    return sample;
}
```

After sign extension, the 24-bit sample is shifted down to signed 16-bit PCM:

```
int16_t sample16 = (int16_t)(sample24 >> 8);
```

This produces `pcm_buffer`, which contains normal signed 16-bit audio samples.

Downsampling for Keyword Spotting

The DMA audio demo records at approximately 96 kHz, while common keyword-spotting models expect 16 kHz input. Therefore, the project reduces the sample rate from 96 kHz to 16 kHz.

Since:

$$96000 / 16000 = 6$$

one 16 kHz output sample is produced from every group of six 96 kHz input samples. The current implementation uses an averaging approach:

```
sum = pcm[i] + pcm[i+1] + pcm[i+2] + pcm[i+3] + pcm[i+4] + pcm[i+5];  
out16k[out_count++] = (int16_t)(sum / 6);
```

The result is:

```
int16_t kws_input[16000];
```

This buffer contains one second of 16 kHz mono audio and is passed to `KWS_Run()`.

Keyword Spotting Module

The current keyword-spotting module is located in:

```
kws.cc  
kws.h
```

The interface is:

```
void KWS_Init(void);  
void KWS_Run(const int16_t *audio_16k, int num_samples);
```

Because `kws.cc` is compiled as C++ while `demo.c` is compiled as C, `kws.h` uses an `extern "C"` wrapper so the C application can call the C++ keyword-spotting functions.

The current `KWS_Run()` implementation verifies that the input contains 16,000 samples and runs a simple energy-based speech detector. This is a placeholder for a future ML model, but it confirms that the audio pipeline is working correctly.

Current expected output:

```
Quiet recording:
KWS result: silence

Speaking recording:
KWS result: speech present, ready for model
```

Future work will replace the placeholder with a TensorFlow Lite Micro or custom embedded keyword-spotting model.

Interfaces and Peripherals Used

Interface / Peripheral	Purpose
Audio Codec	Captures microphone or line-in audio and supports playback
I2C	Configures the audio codec registers
I2S / Audio Stream	Carries the digital audio sample stream
AXI DMA	Transfers audio samples between streaming hardware and DDR memory
DDR Memory	Stores the DMA audio buffer at <code>MEM_BASE_ADDR</code>
Interrupt Controller	Handles DMA completion events
UART	Prints debug information and detection results
Buttons / User I/O	Starts recording or playback depending on the original demo behavior
LEDs, optional	Can be used as a visual output for keyword detection

Verification and Testing

The project was verified incrementally. Each stage was tested before moving to the next stage.

1. DMA Audio Test

The DMA audio was first tested to confirm that audio functionality could be recorded and played back.

2. DMA Buffer Identification

The audio recording buffer was identified as DDR memory beginning at:

```
MEM_BASE_ADDR
```

The DMA S2MM transfer writes audio samples into this memory region.

3. Raw DMA Word Inspection

Raw DMA words were printed over UART. The output showed values such as:

```
0x00FDDD13
0x00022297
0x00FFDB66
```

This confirmed that the audio samples were stored in the lower 24 bits of each 32-bit DMA word.

4. PCM Conversion Test

After implementing 24-bit sign extension and 16-bit PCM conversion, quiet and speaking recordings were compared.

Example results:

```
Quiet recording:
avg_abs ≈ 10 to 14

Speaking recording:
avg_abs ≈ 478 to 505
```

This showed that the converted `pcm_buffer` responded clearly to speech.

5. 16 kHz KWS Buffer Test

After downsampling, the one-second `kws_input` buffer was tested.

Example results:

```
Quiet KWS input:
min      = -7
max      = 11
avg_abs  = 1

Speaking KWS input:
min      = -3344
max      = 2518
avg_abs  = 601
```

This confirmed that the final keyword-spotting input buffer contained valid audio.

6. Placeholder KWS Test

The placeholder energy detector was tested with quiet and speaking recordings.

Expected and observed results:

```
Quiet recording    → KWS result: silence  
Speaking recording → KWS result: speech present, ready for model
```

How to Build and Run

Required Hardware

- Zybo Z7 development board
- USB cable for programming and UART
- Microphone or audio source connected to the appropriate audio input
- Headphones or speakers, optional, for playback testing
- Host computer with Vitis installed

Required Software

- AMD/Xilinx Vitis
- Vivado project and exported hardware platform from the Zybo Z7 DMA Audio Demo
- Serial terminal such as Vitis Serial Terminal, PuTTY, Tera Term, or similar

Build Steps

1. Clone this repository.
2. Open the Vitis workspace.
3. Verify that the hardware platform and BSP are correctly linked.
4. Confirm that `kws.cc` is included in the application project source folder.
5. Confirm that the C++ compiler uses the same ARM flags as the rest of the project:

```
-mcpu=cortex-a9 -mfpv=vfpv3 -mfloat-abi=hard
```

6. Build the application project.
7. Program the FPGA if needed.
8. Run the application on the Zybo Z7.
9. Open the UART terminal.
10. Start a recording using the demo controls.
11. Observe the UART output.

Expected Runtime Output

For a quiet room:

```
KWS result: silence
```

For a spoken recording:

```
KWS result: speech present, ready for model
```

Project Demo Description

The project demo should show:

1. The Zybo Z7 board connected to the host computer.
2. The Vitis application running through UART.
3. A quiet recording that prints `KWS result: silence`.
4. A speaking recording that prints `KWS result: speech present, ready for model`.
5. Optional playback from the original DMA audio demo to confirm audio capture.
6. Explanation of the preprocessing pipeline from raw DMA samples to `kws_input[16000]`.

Current Limitations

- The current implementation performs voice activity detection, not full keyword classification.
- TensorFlow Lite Micro or a custom ML inference engine has not yet been fully integrated.
- The downsampling method uses simple averaging over six samples; a stronger anti-aliasing filter could improve audio quality.
- The project currently processes a recorded one-second window after DMA capture rather than running continuous real-time streaming inference.

Future Work

Planned improvements include:

- Integrate TensorFlow Lite Micro into the Vitis application.
- Use the default Micro Speech model to classify `silence`, `unknown`, `yes`, and `no`.
- Train and deploy a custom keyword model.
- Add real-time ping-pong DMA buffering.
- Add LED or GPIO output when the target keyword is detected.
- Improve downsampling with a better low-pass filter.
- Add confidence thresholds and smoothing over multiple windows.

Repository Contents

Recommended repository structure:

```
README.md
Vivado_Project/
Vitis_Project/
Documentation/
  Final_Report.pdf
  Presentation.pptx
```

```
Demo_Video_Link.txt
src/
demo.c
audio.c / audio.h
dma.c / dma.h
iic.c / iic.h
intc.c / intc.h
userio.c / userio.h
kws.cc / kws.h
```

References

1. Digilent, Zybo Z7 project resources.
2. AMD/Xilinx, Vitis development environment documentation.
3. TensorFlow Lite Micro documentation and Micro Speech example.